



Faculty of Technology
University of Sri Jayewardenepura

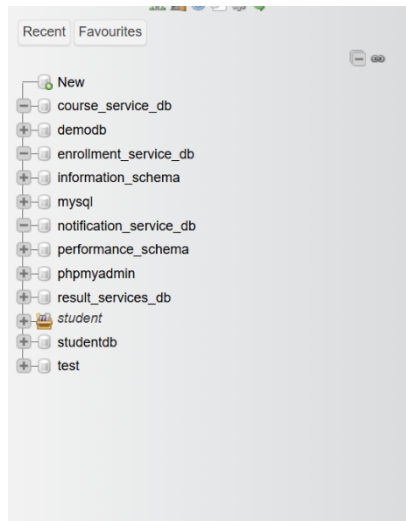
Lab Assessment

Microservices and Cloud Computing

Name: K.P.V. Perera

Index Number: ICT/21/897

Database

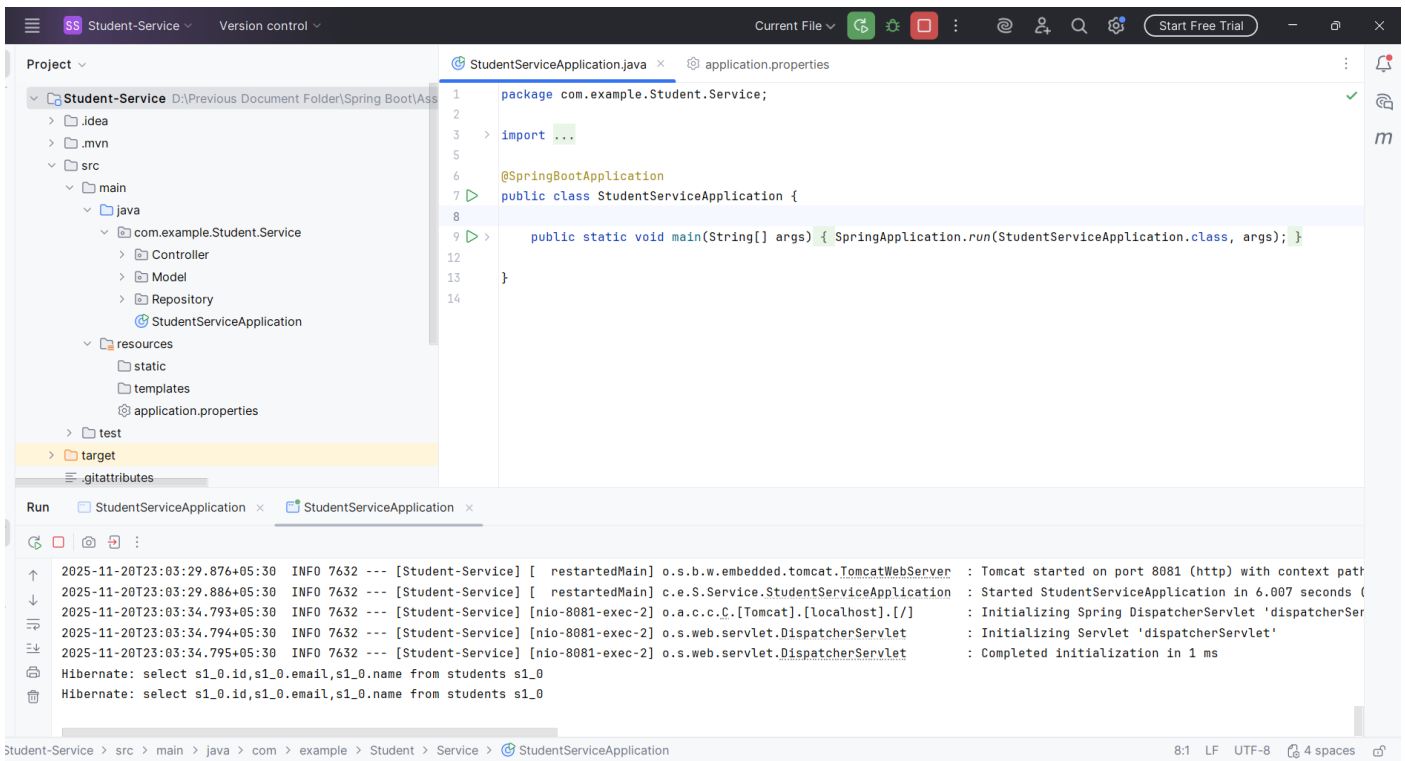


☐ Check all

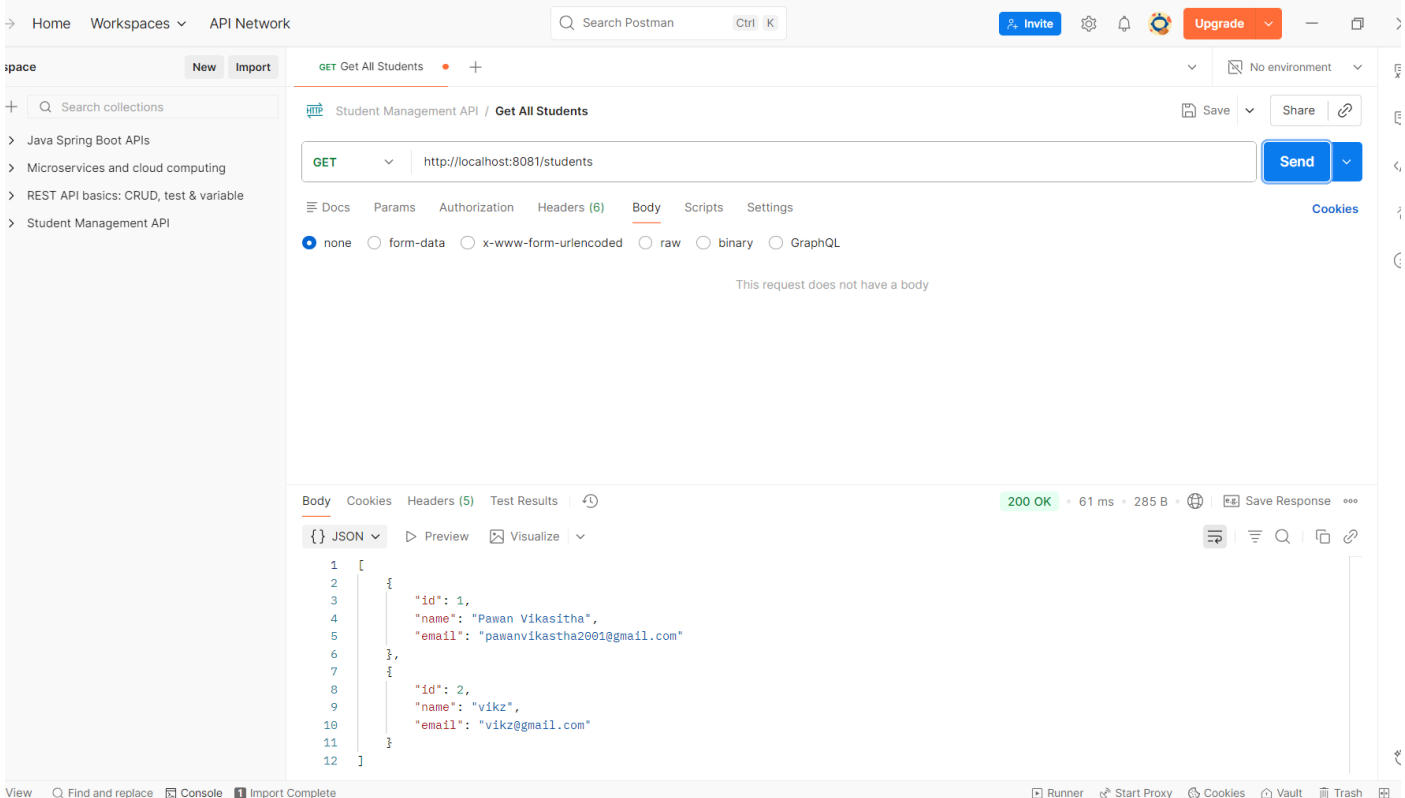
Database	Collation	Action
☐ course_service_db	utf8mb4_general_ci	Check privileges
☐ demodb	utf8mb4_general_ci	Check privileges
☐ enrollment_service_db	utf8mb4_general_ci	Check privileges
☐ information_schema	utf8_general_ci	Check privileges
☐ mysql	utf8mb4_general_ci	Check privileges
☐ notification_service_db	utf8mb4_general_ci	Check privileges
☐ performance_schema	utf8_general_ci	Check privileges
☐ phpmyadmin	utf8_bin	Check privileges
☐ result_services_db	utf8mb4_general_ci	Check privileges
☐ studentdb	utf8mb4_general_ci	Check privileges
☐ student_management_db	utf8mb4_general_ci	Check privileges
☐ student_service_db	utf8mb4_general_ci	Check privileges
☐ test	latin1_swedish_ci	Check privileges

Total: 13

1. Student Service



(a) GET /students



(b) POST /students

The screenshot shows the Postman interface for a workspace named 'workspace'. The active collection is 'Student Management API' and the selected request is 'POST Create Student'. The URL is 'http://localhost:8081/students'. The request body is raw JSON:

```
{  "name": "Pereira Pawan",  "email": "pereira@mail.com",  "course": "SFT",  "age": 20}
```

. The response status is '200 OK' with a response time of 106 ms and a body size of 244 B. The response body is JSON:

```
{  "id": 3,  "name": "Pereira Pawan",  "email": "pereira@mail.com",  "course": "SFT",  "age": 20}
```

(c) GET /students/{id}

The screenshot shows the Postman interface for the same workspace. The active collection is 'Student Management API' and the selected request is 'GET Get Student by ID'. The URL is 'http://localhost:8081/students/2'. The response status is '200 OK' with a response time of 9 ms and a body size of 236 B. The response body is JSON:

```
{  "id": 2,  "name": "vikz",  "email": "vikz@gmail.com",  "course": "BICT",  "age": 20}
```

2. Course service

The screenshot shows an IDE window with the project 'Course-service' open. The file 'CourseServiceApplication.java' is selected, showing the following code:

```
1 package com.example.Course.service;
2
3 import ...
4
5
6 @SpringBootApplication
7 public class CourseServiceApplication {
8
9     public static void main(String[] args) { SpringApplication.run(CourseServiceApplication.class, args); }
10
11 }
12
13
14
```

The Run console shows the following logs:

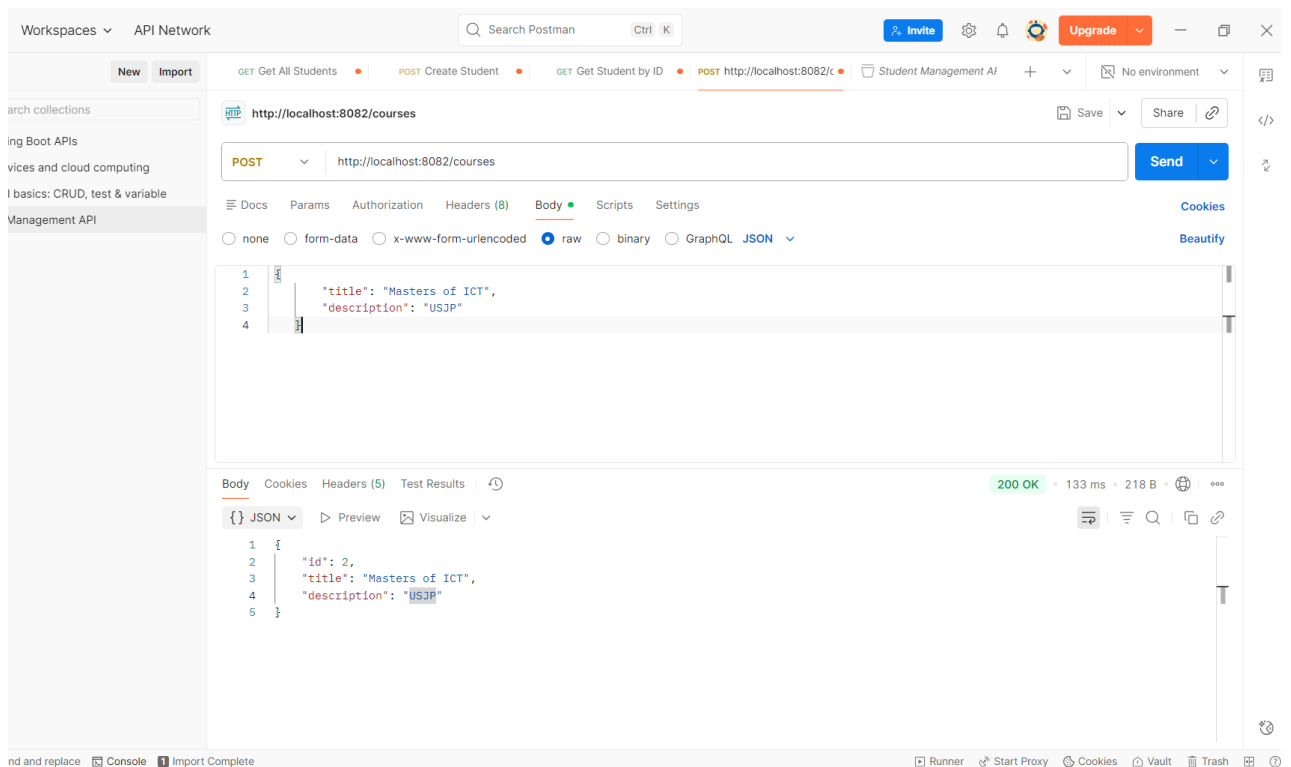
```
2025-11-20T23:18:53.073+05:30 WARN 8968 --- [Course-service] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore,
2025-11-20T23:18:54.196+05:30 WARN 8968 --- [Course-service] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : Unable to start LiveReload server
2025-11-20T23:18:54.282+05:30 INFO 8968 --- [Course-service] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path
2025-11-20T23:18:54.307+05:30 INFO 8968 --- [Course-service] [ restartedMain] c.e.C.service.CourseServiceApplication : Started CourseServiceApplication in 10.262 seconds (p
2025-11-21T01:06:25.031+05:30 INFO 8968 --- [Course-service] [nio-8082-exec-4] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServ
2025-11-21T01:06:25.033+05:30 INFO 8968 --- [Course-service] [nio-8082-exec-4] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2025-11-21T01:06:25.035+05:30 INFO 8968 --- [Course-service] [nio-8082-exec-4] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

(a) GET /courses

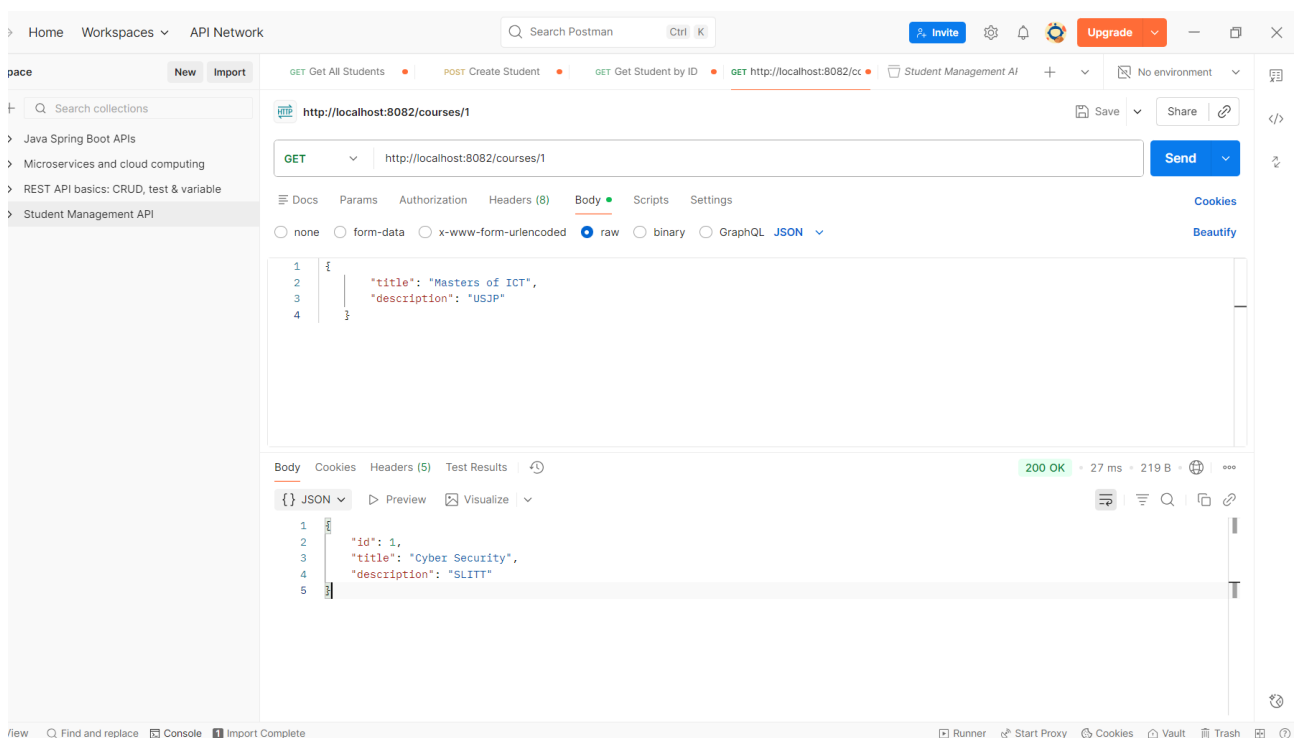
The screenshot shows the Postman interface with a GET request to `http://localhost:8082/courses`. The response is a 200 OK status with a response time of 34 ms and a body size of 221 B. The response body is in JSON format:

```
1 [
2   {
3     "id": 1,
4     "title": "Cyber Security",
5     "description": "SLITT"
6   }
7 ]
```

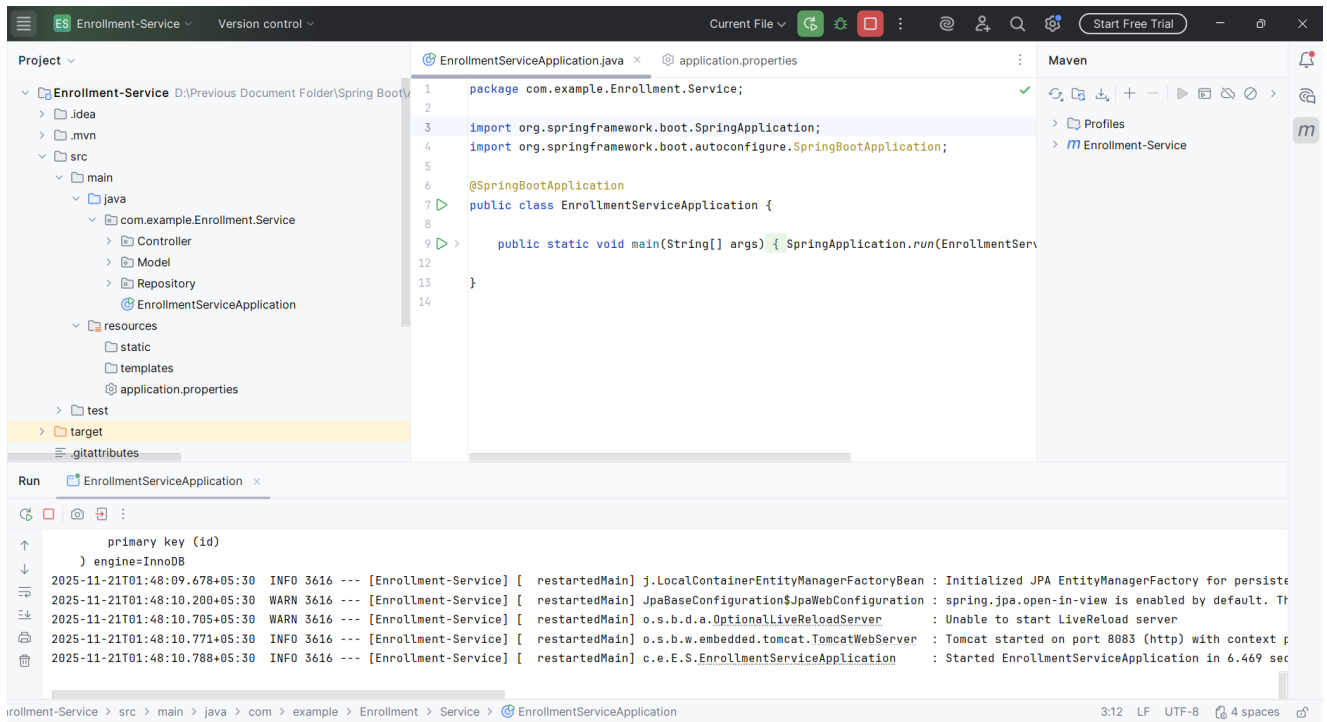
(b) POST /courses



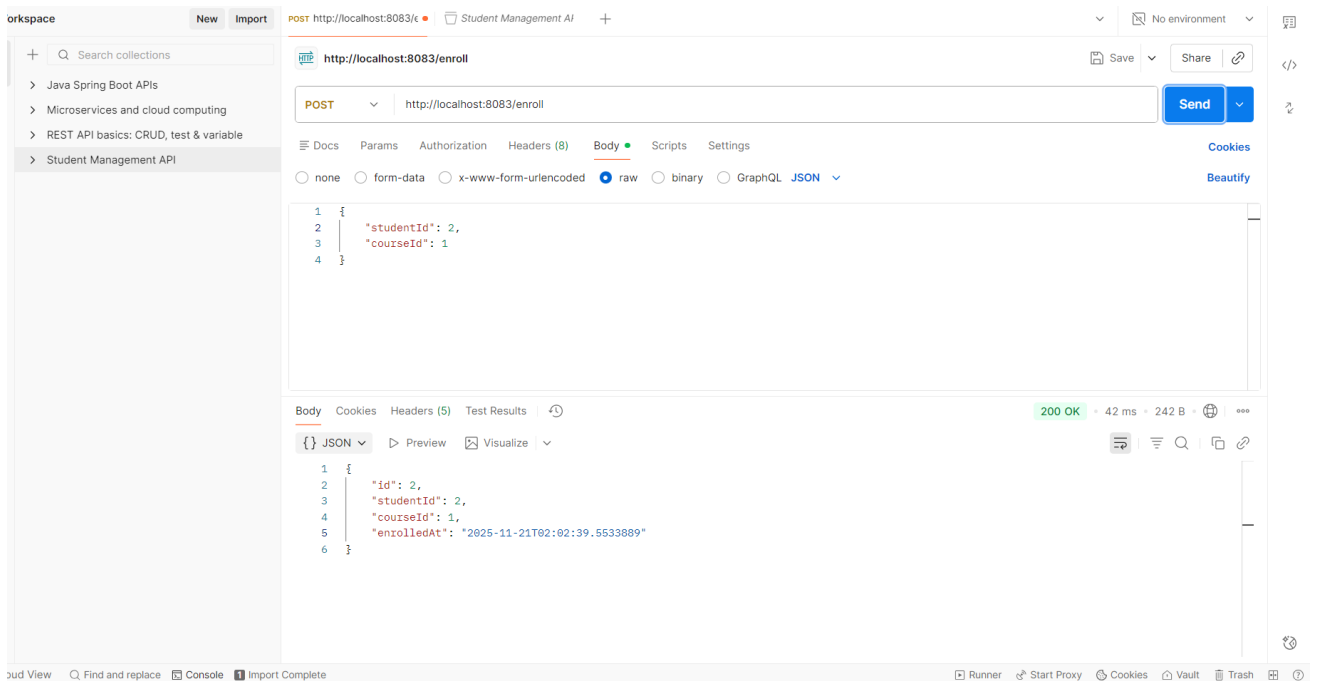
(c) GET /courses/{id}



3. Enrollment Service



(a) POST /enroll (body: studentId, courseId)



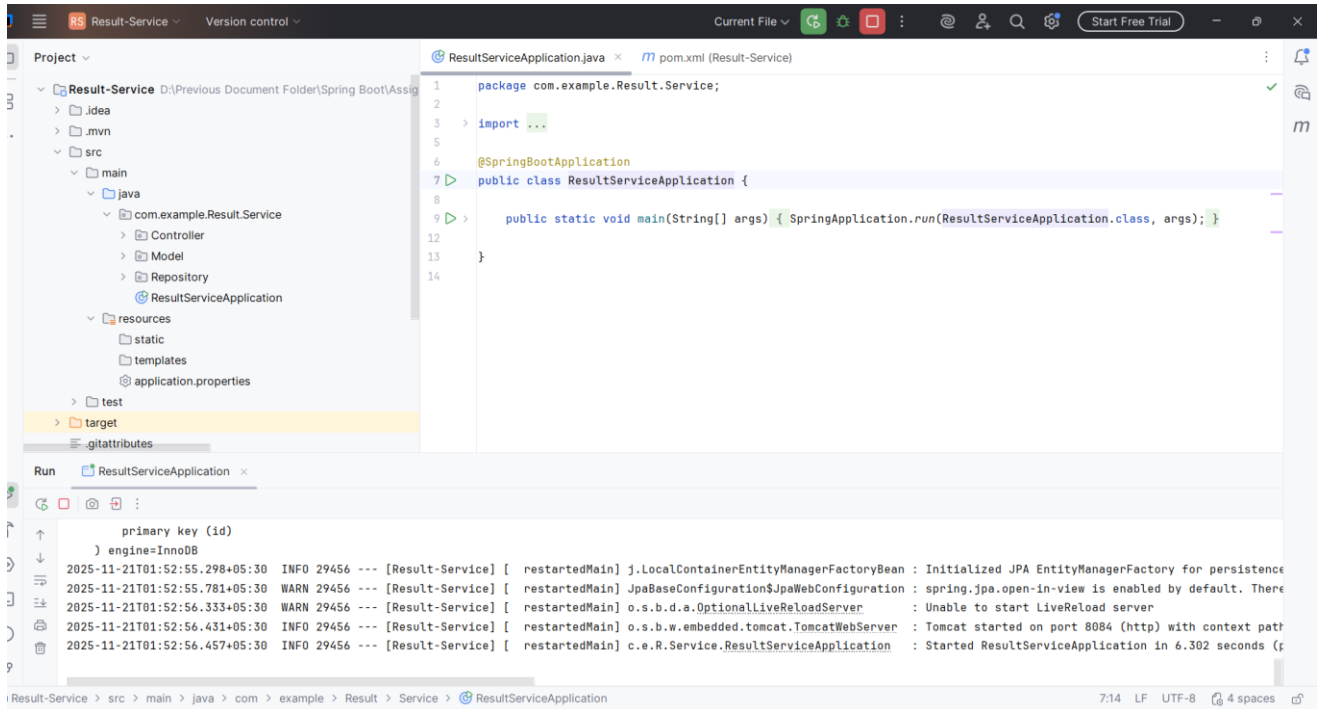
(b) GET /enrollments/student/{id}

The screenshot displays the Postman interface for a REST client. The top bar shows the workspace 'API Network' and a search bar. The left sidebar lists collections, with 'Student Management API' selected. The main area shows a GET request to 'http://localhost:8083/enroll/student/2'. The request is configured with the method 'GET' and the URL 'http://localhost:8083/enroll/student/2'. The 'Body' tab is active, showing a 'raw' body type. The response is displayed in the bottom panel, showing a '200 OK' status with a response time of 21 ms and a body size of 236 B. The response body is a JSON object:

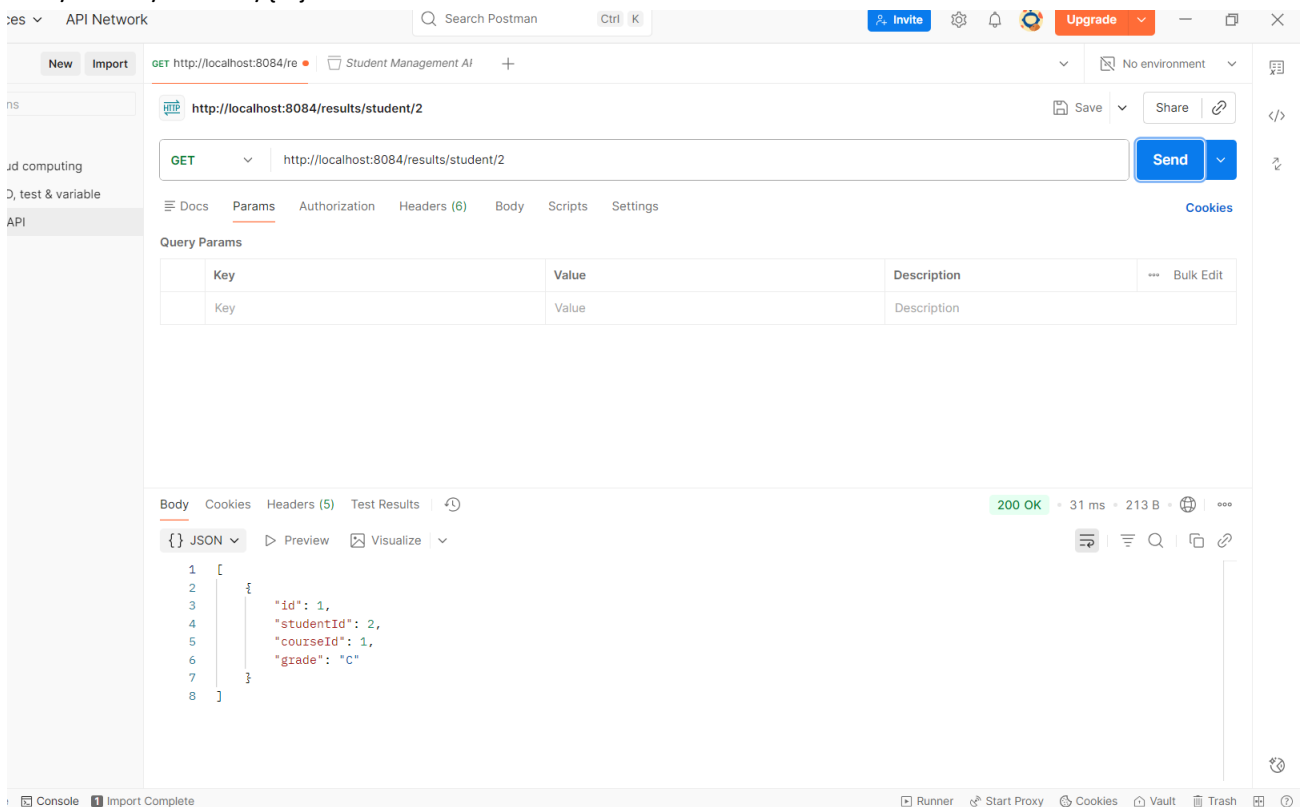
```
1 [
2   {
3     "id": 2,
4     "studentId": 2,
5     "courseId": 1,
6     "enrolledAt": "2025-11-21T02:02:39"
7   }
8 ]
```

The bottom status bar shows 'id View', 'Find and replace', 'Console', and 'Import Complete'.

4. Result Service



(a) GET /results/student/{id}



(b) POST /results (studentId, courseId, grade)

The screenshot displays the Postman interface for a REST client. The workspace is named "API Network". The selected collection is "Student Management API". The request is a POST to "http://localhost:8084/results". The request body is raw JSON, containing the following data:

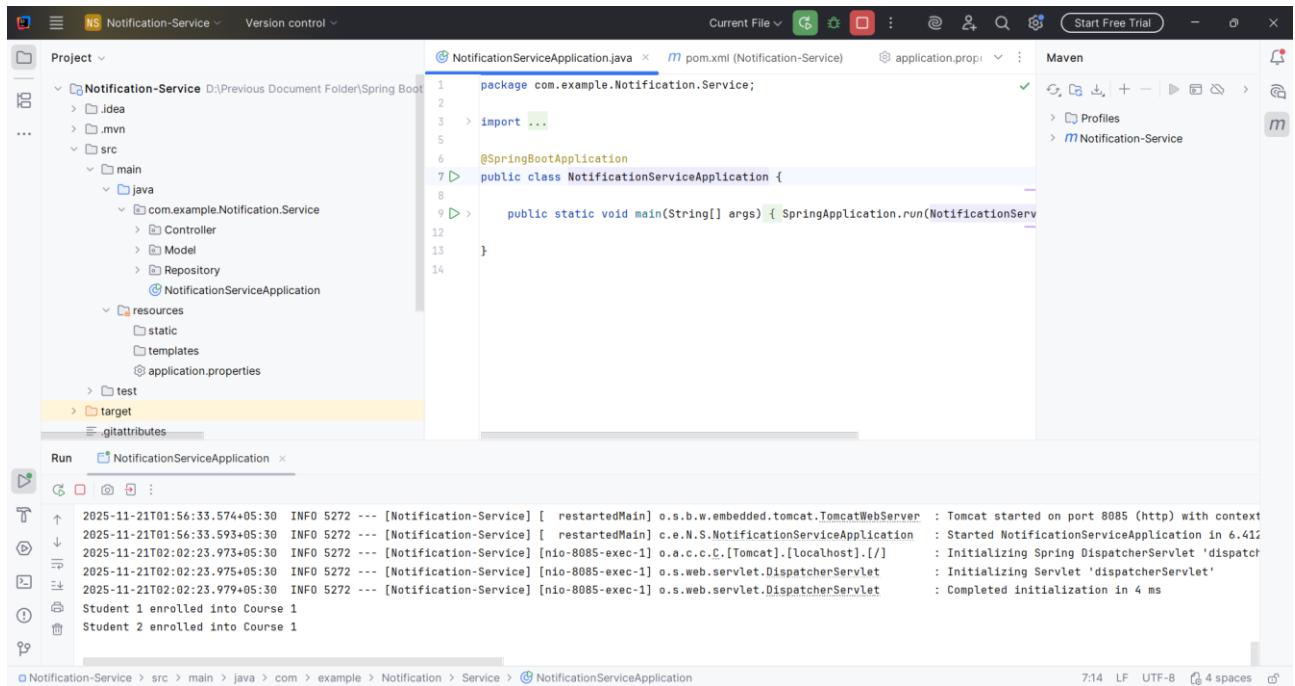
```
1 {
2   "studentId": 1,
3   "courseId": 1,
4   "grade": "B"
5 }
```

The response is a 200 OK status, received in 137 ms with 211 B of data. The response body is JSON, containing the following data:

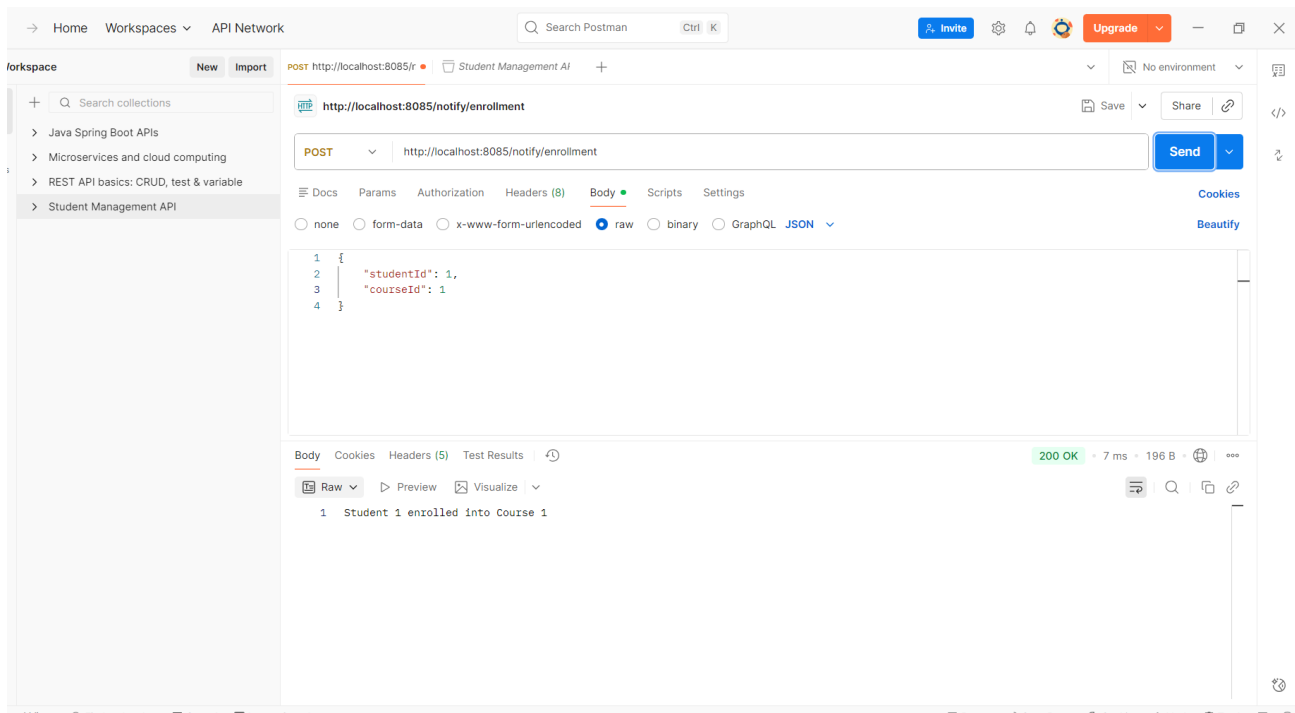
```
1 {
2   "id": 2,
3   "studentId": 1,
4   "courseId": 1,
5   "grade": "B"
6 }
```

The interface also shows tabs for Docs, Params, Authorization, Headers (8), Body, Scripts, and Settings. The Body tab is currently selected, showing the raw JSON body. The response status is 200 OK, and the response body is JSON.

5. Notification Service



(a) POST /notify/enrollment



All Services

The screenshot displays three overlapping IntelliJ IDEA windows, each representing a different microservice in a Spring Boot application.

- Student-Service (Top Left):** Shows the project structure with a package `com.example.Student.Service`. The `StudentServiceApplication` class is visible, containing a `@SpringBootApplication` annotation and a `public class Student` definition.
- Course-service (Top Right):** Shows the `CourseServiceApplication` class. The run console displays the following logs:

```
2025-11-21T01:06:25.035+05:30 INFO 8968 --- [Course-service] [nio-8082-exec-4]
Hibernate: select c1_0.id,c1_0.description,c1_0.title from courses c1_0
Hibernate: select c1_0.id,c1_0.description,c1_0.title from courses c1_0
Hibernate: insert into courses (description,title) values (?,?)
Hibernate: select c1_0.id,c1_0.description,c1_0.title from courses c1_0 where c1_0.id=?
```
- Enrollment-Service (Bottom):** Shows a SQL query in the Maven tool window:

```
select
  e1_0.course_id,
  e1_0.enrolled_at,
  e1_0.student_id
from
  enrollments e1_0
where
  e1_0.student_id=?
```

The bottom status bar of the IDE shows the current file paths for each window: `Result-Service > src > main > java > com > example > Result`, `Enrollment-Service > src > main > java > com > example > Enroll`, and `Notification-Service > src > main > java > com > example > Notif`.